

Ordner-Berater v3

Einzeldatei-Verarbeitung mit Metadaten

Bedienungsanleitung & Architektur

Lokale LLM-basierte Dateibenennung und Metadaten-Anreicherung
Qwen3-27B · LM Studio · Gradio · Python

Stand: 17. Mai 2026

Inhaltsverzeichnis

Teil 1: Bedienungsanleitung	3
1.1 Voraussetzungen.....	3
1.2 LM Studio einrichten.....	3
1.3 Konfiguration im Notebook	4
1.4 Workflow: Schritt für Schritt	5
Teil 2: Architektur	6
2.1 Drei-Kanal-Architektur	6
2.2 Zellen-Struktur des Notebooks	6
2.3 LLM-Client-Reset.....	6
2.4 Metadaten-Schreiber	7
2.5 Checkpoint- und Undo-System.....	7
Teil 3: Skills-System.....	8
3.1 Verzeichnisstruktur	8
3.2 Zusammenspiel: Prompts und Profile.....	8
3.3 Mitgelieferte Profile.....	8
3.4 Eigene Profile erstellen.....	9
3.5 Prompts anpassen.....	9
Anhang.....	10
A. Unterstützte Dateitypen.....	10
B. Fehlerbehandlung	10
C. Bekannte Einschränkungen	10

Teil 1: Bedienungsanleitung

Diese Anleitung beschreibt die Einrichtung und Verwendung des Ordner-Beraters v3 — eines lokal laufenden Werkzeugs, das mithilfe eines LLM (Qwen3-27B) Dateien automatisch umbenennt und mit Stichwort-Metadaten versieht.

1.1 Voraussetzungen

Software

Komponente	Version	Zweck
Python	3.11+	Laufzeitumgebung
LM Studio	0.4.x	Lokaler LLM-Server
Qwen3.6-27B	Dense, multimodal	LLM (Text + Vision)
Jupyter Notebook	beliebig	Notebook-Umgebung

Python-Bibliotheken

Werden beim ersten Start automatisch installiert, falls nicht vorhanden:

```
pip install gradio openai python-docx python-pptx openpyxl PyMuPDF Pillow beautifulsoup4 stripptf odfpy xlrd
```

Hardware

Der Ordner-Berater ist für lokale GPU-Inferenz optimiert. Mindestanforderung: eine GPU mit 16 GB VRAM (z.B. RTX 3080 Ti). Optimal: zwei GPUs mit zusammen 40 GB (z.B. RTX 3090 + RTX 3080 Ti im eGPU-Setup, in LM Studio gekoppelt).

1.2 LM Studio einrichten

Die korrekte LM-Studio-Konfiguration ist entscheidend für stabile Ergebnisse über hunderte Dateien.

Modell laden

In LM Studio das Modell **Qwen3.6-27B** (oder einen vergleichbaren Qwen3-Coder) laden. Beim Laden die folgenden Einstellungen prüfen — dazu unten im Lade-Dialog „**Show advanced settings**“ aktivieren:

Einstellung	Empfehlung	Begründung
Max Concurrent Predictions	1	Das Skript sendet immer nur eine Anfrage gleichzeitig. Der Default 4 allokiert KV-Cache für 4 parallele Slots, von denen 3 ungenutzt VRAM verschwenden.
Flash Attention	Ein (An)	Reduziert VRAM-Verbrauch für den KV-Cache erheblich und beschleunigt die Inferenz um ein Vielfaches.
Context Length	8.192	Die Prompts sind kurz (< 2.000 Tokens). Ein kleineres Context-Window spart direkt VRAM. Größer nur nötig bei extrem langen Textextrakten.
GPU Offload	Max	Alle Layer auf die GPU — CPU-Inferenz ist um Größenordnungen langsamer.

Einstellung	Empfehlung	Begründung
Offload KV Cache to GPU	Ein (An)	KV-Cache auf der GPU halten für maximale Geschwindigkeit.
Unified KV Cache	Egal bei 1 Slot	Nur relevant bei mehreren Concurrent Predictions.

Wichtig: Max Concurrent Predictions auf 1

Dies ist die häufigste Ursache für Qualitätsdegradation bei langen Läufen. Bei 4 Slots (Default) wird 4x so viel VRAM für den KV-Cache reserviert. Bei knappem VRAM produziert das Modell schlechtere Ergebnisse, bevor es sichtbar abstürzt.

Server-Einstellungen

Im Reiter „**Developer**“ den lokalen Server aktivieren. Der Ordner-Berater verbindet sich standardmäßig auf:

```
http://localhost:1234/v1
```

Parameter	Empfehlung	Begründung
Temperature	0.2	Wird vom Skript per API gesetzt. Niedrig für konsistente, reproduzierbare Dateinamen.
Thinking-Modus	Aus	Qwen3 erzeugt sonst <think>-Blöcke vor dem JSON-Output. Das erhöht die Token-Anzahl unnötig und stört gelegentlich das JSON-Parsing.
Max Tokens	1.024	Wird vom Skript gesetzt. Reicht für Name + Stichwörter + JSON.

Tipp: Thinking-Modus deaktivieren

Qwen3-Modelle haben einen integrierten Thinking-Modus, der über das Chat-Template gesteuert wird. In LM Studio kann man diesen unter den Chat-Einstellungen („Chat Preset“) deaktivieren oder im Prompt /no_think verwenden. Das Skript versucht Thinking-Blöcke im Output zu entfernen, aber Prävention ist robuster.

1.3 Konfiguration im Notebook

Alle konfigurierbaren Parameter befinden sich in **Zelle 2 „Konfiguration“** des Notebooks:

Variable	Default	Beschreibung
LM_STUDIO_URL	localhost:1234/v1	API-Endpunkt von LM Studio
MODELL_NAME	qwen3-coder	Modellname wie in LM Studio angezeigt
LLM_RESET_INTERVALL	50	Nach N Dateien wird der LLM-Client neu erstellt, um KV-Cache-Probleme zu vermeiden
PROFIL_NAME	lehramt	Aktives Profil: lehramt, allgemein, forschung
SKILLS_VERZEICHNIS	ordner_berater_skills	Pfad zu den Skill-Dateien (relativ oder absolut)
CHECKPOINT_Pfad	~/ordner_berater_checkpoint.json	Fortschritt wird nach jeder Datei gespeichert
UNDO_LOG_Pfad	~/ordner_berater_undo.json	Alle Umbenennungen werden protokolliert

1.4 Workflow: Schritt für Schritt

Schritt 0: Bildformate konvertieren (optional)

Konvertiert GIF, BMP, WEBP, ICO und TIFF in PNG-Dateien, damit sie Metadaten empfangen können. Nur beim ersten Durchlauf nötig. Die Originaldateien werden gelöscht, die Konvertierung im Undo-Log protokolliert.

Schritt 1: Ordner scannen

Einen Ordnerpfad eingeben und „**Scannen**“ klicken. Der Scanner durchläuft den Ordner rekursiv und:

- erfasst alle Dateien mit Metadaten (Größe, Typ, Änderungsdatum, SHA-256-Hash)
- extrahiert Titel und Inhaltsauszüge aus 49 Dateitypen (Office, PDF, Code, Text, HTML...)
- zählt eingebettete Bilder in PDF, DOCX und PPTX
- erkennt Duplikate, generische Namen und Versionierungsmuster

Das Ergebnis ist eine Zusammenfassung mit Dateitypverteilung, Ordnerstruktur und auffälligen Dateien.

Schritt 2: Einzeldatei-Verarbeitung

Das Herzstück: Jede Datei wird einzeln an das LLM geschickt. Per Dropdown kann vor dem Start das **Profil** gewählt werden (Lehramt, Allgemein, Forschung). Für jede Datei geschieht Folgendes:

1. Kanal-Auswahl (Text / Hybrid / Vision) basierend auf Dateityp und verfügbarem Inhalt
2. LLM-Call: Das Modell erhält den System-Prompt (aus dem gewählten Profil) und den Dateiinhalt
3. JSON-Parsing: Der vorgeschlagene Name und 5–10 Stichwörter werden extrahiert
4. Umbenennung: Wenn der neue Name sich unterscheidet und valide ist
5. Metadaten: Stichwörter werden direkt in die Datei geschrieben (Keywords-Feld)
6. Checkpoint: Status wird nach jeder Datei gespeichert

Fortsetzen nach Unterbrechung

Das Checkpoint-System speichert den Bearbeitungsstand nach jeder einzelnen Datei. Bei einem Absturz oder manuellem Abbruch kann die Verarbeitung nahtlos fortgesetzt werden — bereits verarbeitete Dateien werden übersprungen. Den Checkpoint kann man über den Button „Checkpoint löschen“ zurücksetzen.

Schritt 3: Übriggebliebene markieren

Dateien, die nicht erfolgreich verarbeitet wurden (Binärdateien, fehlgeschlagene Extraktion, nicht erkannte Bilder), werden identifiziert und können mit einem Präfix markiert werden:

Präfix	Bedeutung
binaer	ML-Modelle, Archive, ausführbare Dateien
_bild_fehler_	Bilder, die das Vision-LLM nicht erkennen konnte
_text_fehler_	Textdateien mit fehlgeschlagener Extraktion
manuell	Kein Extraktor vorhanden

So sind nach der Verarbeitung alle Dateien entweder sinnvoll benannt oder klar als manuell zu prüfen markiert.

Teil 2: Architektur

Dieser Teil beschreibt den technischen Aufbau des Skripts für Entwickler und Dozenten, die den Code verstehen oder erweitern möchten.

2.1 Drei-Kanal-Architektur

Das Skript wählt pro Datei automatisch den optimalen Verarbeitungspfad:

Kanal	Auslöser	LLM-Input	Token-Budget
Text-only	Hat Textextrakt, keine Bilder	Dateiname + Titel + Extrakt	~500–800
Hybrid	Textextrakt UND eingebettete Bilder	Text + max. 3 größte Bilder	~2.000–4.000
Vision-only	Standalone-Bild oder Scan-PDF	Bilddatei oder gerenderte Seite 1	~1.000–2.000

Die Entscheidungslogik im Dispatcher (**einzeldatei_verarbeiten**):

- Ist es ein Bild (PNG, JPG...)? → **Vision-only**
- Hat es Text UND eingebettete Bilder? → **Hybrid** (Bilder werden aus PDF/DOCX/PPTX extrahiert)
- Hat es nur Text? → **Text-only**
- Ist es ein PDF ohne Text? → Seite 1 rendern → **Vision-Fallback**
- Alles andere → überspringen

2.2 Zellen-Struktur des Notebooks

Das Notebook ist in 8 aufeinander aufbauende Zellen gegliedert:

Zelle	Inhalt	Abhängigkeiten
1	Abhängigkeiten installieren	—
2	Konfiguration (URLs, Pfade, Reset-Intervall, Profil)	—
3	Scanner + Inhaltsextraktion (49 Dateitypen, Bilderzählung)	Zelle 2
4	LLM-Client, Skill-Loader, Checkpoint, Undo-Log	Zelle 2
5	Metadaten-Schreiber (Office, PDF, JPEG, PNG)	Zelle 3
6	Drei-Kanal-Verarbeitung + Prompt-Loading	Zelle 4, 5
7	Übriggebliebene markieren	Zelle 3
8	Gradio-Interface + Workflow-Steuerung	Alle

2.3 LLM-Client-Reset

Ein bekanntes Problem bei langen Sessions mit lokalen LLMs ist die schleichende Qualitätsdegradation. Der Ordner-Berater begegnet dem mit zwei Maßnahmen:

1. LM Studio-seitig: Max Concurrent Predictions auf 1 setzen (siehe Abschnitt 1.2). Damit wird nur ein KV-Cache-Slot allokiert, und der VRAM-Druck sinkt.

2. Skript-seitig: Nach jedem LLM-Call wird ein Zähler erhöht. Erreicht er den Wert von **LLM_RESET_INTERVALL** (Standard: 50), wird der OpenAI-Client neu erstellt. Das erzwingt eine frische HTTP-Session. Das Prefix-Caching in llama.cpp (Wiederverwendung des System-Prompt-KV-Cache) funktioniert weiterhin — es basiert auf Token-Matching, nicht auf der HTTP-Verbindung.

2.4 Metadaten-Schreiber

Stichwörter werden direkt in die Dateien geschrieben — nicht in eine externe Datenbank. So bleiben die Metadaten auch beim Kopieren oder Verschieben der Dateien erhalten.

Format	Feld	Methode
.docx / .pptx / .xlsx	core_properties.keywords	python-docx / python-pptx / openpyxl
.pdf	metadata.keywords + subject	PyMuPDF (inkrementelles Speichern)
.jpg / .jpeg	EXIF Tag 270 + 40092	Pillow (quality='keep', kein Qualitätsverlust)
.png	tEXt-Chunks	Pillow PngInfo (verlustfrei)

Bestehende Metadaten werden ergänzt, nicht überschrieben. Das EXIF-Tag 40092 (XPKeywords) macht JPEG-Stichwörter im Windows-Explorer durchsuchbar.

2.5 Checkpoint- und Undo-System

Checkpoint: Nach jeder verarbeiteten Datei wird der vollständige Status (alter Name, neuer Name, Stichwörter, Kanal, Metadaten-Status) in eine JSON-Datei geschrieben. Bei Neustart werden bereits verarbeitete Dateien übersprungen.

Undo-Log: Jede Umbenennung wird mit Zeitstempel, Quell- und Zielpfad im Undo-Log protokolliert. Dieses Log ermöglicht eine manuelle Rückgängigmachung — eine automatische Undo-Funktion ist als Erweiterung denkbar.

Teil 3: Skills-System

Die Systemprompts sind nicht mehr im Python-Code hartcodiert, sondern als Markdown-Dateien ausgelagert. Das ermöglicht schnelle Iteration ohne Code-Änderungen und vereinfacht die Anpassung an neue Anwendungsfälle.

3.1 Verzeichnisstruktur

```
ordner_berater_skills/
├── prompts/                # Prompt-Templates (3 Kanäle)
│   ├── text_benennung.md   # Text-only-Kanal
│   ├── hybrid_benennung.md # Text + eingebettete Bilder
│   └── vision_benennung.md # Standalone-Bilder / Scan-PDFs
└── profile/                # Domänenspezifisches Wissen
    ├── lehramt.md          # Fachkürzel, Typ-Präfixe für Schule
    ├── allgemein.md        # Generische Regeln
    └── forschung.md        # Akademisches Benennungsschema
```

3.2 Zusammenspiel: Prompts und Profile

Jede Prompt-Datei enthält einen Platzhalter **{PROFIL}**, der zur Laufzeit durch den Inhalt der gewählten Profil-Datei ersetzt wird:

Prompt-Template (text_benennung.md)	Profil (lehramt.md)
<pre>Du bist ein Dateibenennungs- Experte... {PROFIL} AUSGABE: Nur JSON...</pre>	<pre>TYP-PRÄFIXE: AB = Arbeitsblatt KA = Klassenarbeit EWH = Erwartungs- horizont FACHKÜRZEL: INF, CH...</pre>

So kann man das gleiche Prompt-Template mit verschiedenen Profilen verwenden, ohne die Prompts selbst zu duplizieren.

3.3 Mitgelieferte Profile

Lehramt (lehramt.md)

Optimiert für Unterrichtsmaterialien eines Lehramt-Studiengangs. Definiert Typ-Präfixe wie AB (Arbeitsblatt), KA (Klassenarbeit), EWH (Erwartungshorizont), Praes (Präsentation) sowie Fachkürzel (INF, CH, PH, NWT). Stichwörter zielen auf Klassenstufe, Fachbegriffe, Methodik/Didaktik und Bildungsplan-Bezug.

Allgemein (allgemein.md)

Generisches Profil für beliebige Ordnerstrukturen. Typ-Präfixe: Dok, Praes, Tab, Bericht, Notiz, Brief, Vertrag, Rechnung, Protokoll usw. Keine Fachkürzel — stattdessen wird das Thema direkt im Namen verwendet. Stichwörter orientieren sich an Projekt, Fachbegriffen, Dokumentstatus und Zeitbezug.

Forschung (forschung.md)

Für akademische Arbeitsumgebungen. Typ-Präfixe: Paper, Review, Thesis, Proposal, Poster, Analyse, Lit usw. Stichwörter fokussieren auf Forschungsgebiet, Methodik, Konferenz/Journal, Koautoren und Publikationsstatus.

3.4 Eigene Profile erstellen

Um ein neues Profil hinzuzufügen, genügt eine einzelne Markdown-Datei im Verzeichnis **profile/**. Die Datei sollte drei Abschnitte enthalten:

- 1. TYP-PRÄFIXE:** Eine Liste der verwendeten Kürzel mit Beschreibung. Das LLM wählt das passendste.
- 2. FACHKÜRZEL:** Optional. Kürzel, die dem Dateinamen vorangestellt werden (z.B. INF=Informatik). Wenn nicht gewünscht: „Nicht verwenden“ schreiben.
- 3. STICHWORT-KATEGORIEN:** Welche Arten von Stichwörtern das LLM generieren soll (z.B. Klassenstufe, Methodik, Fachbegriffe).

Beispiel: Profil für eine Anwaltskanzlei

Eine neue Datei `ordner_berater_skills/profile/kanzlei.md` erstellen mit Typ-Präfixen wie Vertrag, Schriftsatz, Gutachten, Mandant, Akte und Stichwort-Kategorien wie Rechtsgebiet, Aktenzeichen, Mandantenname, Frist, Gerichtsbarkeit. In der Konfiguration `PROFIL_NAME = "kanzlei"` setzen oder im Gradio-Dropdown auswählen.

3.5 Prompts anpassen

Die Prompt-Templates in **prompts/** können ebenfalls editiert werden. Der Platzhalter `{PROFIL}` muss erhalten bleiben. Typische Anpassungen:

- **JSON-Schema ändern:** Andere Felder im Output verlangen (z.B. Konfidenz, Sprache)
- **Few-Shot-Beispiele:** Konkrete Input→Output-Paare in den Prompt einfügen
- **Benennungsregeln:** Andere Maximal-Länge, anderes Format (z.B. YYYY-MM-DD_Thema)
- **Anti-Halluzinations-Regeln:** Verstärken, wenn das Modell Themen erfindet

Achtung beim Bearbeiten

Die Prompts werden bei jedem Verarbeitungsstart frisch geladen. Man kann sie also während einer laufenden Gradio-Session editieren und beim nächsten Klick auf „Verarbeitung starten“ sofort testen. Ein Notebook-Neustart ist nicht nötig.

Anhang

A. Unterstützte Dateitypen

Der Ordner-Berater kann aus **49 Dateitypen** Inhalte extrahieren. Eingebettete Bilder werden aus PDF, DOCX und PPTX extrahiert. Metadaten können in 7 Formate geschrieben werden (DOCX, PPTX, XLSX, PDF, JPG, JPEG, PNG).

B. Fehlerbehandlung

Das Skript ist auf Robustheit ausgelegt:

- **JSON-Parsing:** Ein mehrstufiger Fallback extrahiert JSON auch aus Thinking-Blöcken und Markdown-Fencing
- **Dateizugriff:** PermissionError und OSError werden pro Datei gefangen, nie für den ganzen Lauf
- **Hybrid-Fallback:** Wenn die Bildextraktion fehlschlägt, wird automatisch der Text-only-Kanal verwendet
- **Vision-Fallback:** PDFs ohne Text und ohne extrahierbare Bilder werden als Bild gerendert (Seite 1)
- **Metadaten:** Fehlgeschlagenes Schreiben wird protokolliert, unterbricht aber nicht die Verarbeitung

C. Bekannte Einschränkungen

- **Kein automatisches Undo:** Das Undo-Log existiert, muss aber manuell ausgewertet werden
- **Kein paralleles Processing:** Dateien werden sequenziell verarbeitet (bewusste Entscheidung für Stabilität)
- **Keine Duplikat-Auflösung:** Duplikate werden erkannt, aber nicht automatisch behandelt
- **VRAM-Limit:** Bei sehr großen Bildern (> 10 MB) oder langen PDFs kann der Vision-Kanal an VRAM-Grenzen stoßen